

wikigen

**Private data can become verified
reward without becoming public
data.**

dnai-wikigen is a private evaluation, private reward, and attested settlement substrate: sealed data and verifiers inside dstack TEEs, bounded outputs and escrow on Base.

project pitch, not a startup pitch

THE PROBLEM

The most valuable data cannot be shared like ordinary data.

Bio datasets, private benchmarks, reward functions, source accounts, and proprietary tests have value because they can evaluate something. Direct disclosure destroys that leverage.

- Data owners need proof of utility without handing over the data.
- Optimizers need reward signals — but raw rewards, logs, and checkpoints can leak the verifier.
- Buyers need evidence the evaluation actually ran: measured code and a quote, not a PDF promise.

Interaction with private data is not the same as disclosure.

That is only true under a constrained interface: an allowed query family, measured code, authenticated callers, budgeted and reduced outputs, and an attestable transcript.

sealed asset

Raw data, reward functions, account credentials, and browser sessions live only inside the attested TEE.



reward oracle

Measured verifier code computes exact rewards over the sealed data — a narrow oracle, not a shell or notebook.



bounded proof

Only score bands, pass/hold/deny decisions, hashes, quotes, and settlement events leave the boundary.

A private verified-reward substrate: four ideas in one boundary.

NDAI economics

Reserve price, budget cap, bounded disclosure, and escrow settlement mediate every deal.

TEE custody

Data, credentials, reward functions, and browser sessions are sealed in dstack Intel TDX CVMs — operators get no raw access.

private verified rewards

Rewards come from measured code over sealed data, not from a free-form model opinion.

DevProof verification

Anyone can check the chain:
git SHA → image digest →
compose hash → TDX quote →
contract.

Two product faces share one core: the Attested Diligence Room values private artifacts; the Private Reward Lab lets optimizers actively test candidates against sealed data.

One protected interface, four product modes.

A · Diligence Room

private artifact → bounded valuation → escrow settlement

Price memos, code, datasets, SOPs, and model artifacts without disclosure.

B · Private Reward Lab

sealed verifier + data → reward oracle → optimizer loop → bounded proof

RLVR, TTT-style discovery, private benchmarks, reward-function licensing.

C · Source Custody Room

web2 account → TEE browser/export → sealed artifact → policy-gated use

WhatsApp, email, private repos, and data portals become sealed assets.

D · Multi-Owner Collaboration

N private corpora → fan-out gates → unanimous or M-of-N consent → joint result

Biobank collaboration, sponsor/CRO workflows, cross-institution analysis.

The Private Reward Lab is the strongest form of the diligence room: the evaluator does not just read an artifact — it tests candidates against it while the data stays sealed.

Prime RL meets TEEs.

Community-built RL environments become sealed, attested reward oracles — royalty-bearing assets instead of donated public goods.

envs are assets

An environment creator seals data and verifier in a TEE. Anyone can train against it; nobody can copy it.

usage is provable

Every fine-tune that touches the environment is metered by measured code, so royalties are forced by contract, not by trust.

inference pays

The fine-tuned model stays encumbered inside the boundary — payouts stream from those who use the model, not those who trained it.

Bootstrap: the account-delegated Tinker proxy makes this runnable as a DevProof MVP today — no protocol-level integration required.

TTT-Discover-style biology becomes a private reward environment.

Private bio verifier

- Candidate computational-bio code is proposed by an optimizer.
- The TEE scores it against sealed data with hidden train / reward / final-validation splits.
- Exact rewards can drive learning internally; public feedback is quantized, banded, or withheld.
- A one-shot final-validation gate and query budgets stop holdout overfitting.

The optimizer is swappable

RLVR • test-time training • LLM repair loop • evolutionary search • manual

The private reward boundary is the product; the optimizer is a plug-in.

Only the approved reward transcript leaves the TEE.

Private data can be queried many times inside the boundary, but every query passes through one measured interface.

01

Seal D

Private data and verifier code live inside the attested TEE.

02

Run $R_D(c)$

Candidate code is sandboxed and scored against hidden data.

03

Release $Q(\cdot)$

Only reduced feedback leaves: band, hash, hold, or final result.

04

Account for leakage

The public transcript is counted before the system is trusted.

Theorem 1 — leakage-only realization

Every outside view can be simulated from $L(D)$, the declared leakage. If a fact is not in $L(D)$, it does not appear outside the TEE — up to TEE, attestation, and standard crypto assumptions.

$$I(D ; \text{transcript}) \leq n \cdot k + b$$

n reward queries $\times$ k visible bits $+ b$ final-result bits

Delegate the compute, never the account.

The upstream Tinker API key, project id, cookies, and payment state are sealed service configuration inside the CVM. Users and agents only ever hold scoped proxy credentials.

01

scoped JWT

Short-lived, signed with CVM-derived key material, delivered encrypted to the recipient's X25519 key.

02

policy gates

Hash-only issue policy, verifier-signed identity registry, reviewer-signed grant lifecycle — all fail closed.

03

spend limits

Per-scope caps enforced before automation, plus on-chain TinkerAccountEncumbrance approval for spend scopes.

04

bounded receipts

Hashes, bands, booleans, and audit records only — `raw_secret_egress=false` on every response.

Live today: encrypted token issuance, recipient-side decrypt, and scoped proxy calls run on the operator-validation Phala CVM, gated by an on-chain-approved compose hash — with only hashes leaving the boundary.

WHAT EXISTS NOW

This is a running system, not a concept deck.

real

Three contracts live on Base Sepolia with full test suites: DiligenceRoom escrow, EmailOracleAuth policy, and TinkerAccountEncumbrance spend caps.

real

Private reward stack: environment contract, Python candidate sandbox, hidden-holdout accounting, and a synthetic environment with a bounded demo CLI.

real

Chain watcher, TEE chain submitter, verifier-signed submitResult(), and a local synthetic-room-to-Anvil settlement proof that emits bounded JSON only.

real

Live Phala CVM: sealed Tinker account with full API-key lifecycle (create / list / delete) via in-TEE browser automation, a \$20 balance, scoped proxy JWTs, and delegated training driven end-to-end through the real /tinker/train endpoint.

partial

Coordination layer: policy kernel, consent decisions, and reviewer queue are source-real; service wiring and production governance remain open.

Everything public is hashes, bands, booleans, and attestations — the bounded-output rule is enforced in code and tests, not just prose.

HONEST CURRENT STATE

The hard parts are named, not hidden.

Deployed Tinker training

the delegated proxy->train path runs end-to-end (verify -> scope -> spend-limit -> create -> forward/backward -> checkpoint -> bounded receipt), proven through the real endpoint; the one gap is upstream compute — a billing flag Tinker set on our funded account, clearable only by Tinker.

Quote verification

verifier signatures and dstack envelope checks are real; full TDX quote parsing and a production verifier service are not.

Account funding

the funded \$20 balance and card on file are real, yet Tinker's account billing flag persists; production custody, compliance approval, and a tokenized payment route remain open.

Bio validation

interfaces and toy environments only; real TTT/RL loops and OS-level candidate isolation are still to build.

Production surface

frontend, DLP/egress enforcement, reviewer governance, and Proof-of-Cloud provenance are roadmap items.

The trust boundary becomes public infrastructure.

- Attestations turn private computation into something users can verify end to end: git SHA → image digest → compose hash → TDX quote.
- Contracts enforce reserve prices, budget caps, spend policy, result hashes, and pull-payment settlement.
- Private rewards create markets for benchmarks, biology, audits, and RL environments — with contract-forced royalties to their creators — without dumping raw assets on-chain.
- The Flashbots / DevProof lesson applies: name every role, freeze the measurements, publish the verification path.

Next work turns the proof model into a demo anyone can verify.

1

Clear the Tinker account billing gate

The delegated training path is proven end-to-end; the one remaining step is Tinker activating our funded account (clear the 402).

2

Production result verifier

Live TDX quote parsing and a deployed-CVM submitResult broadcast on Base Sepolia.

3

Real reward environments

Bio benchmark, hardened OS-level candidate sandbox, and optional DP accounting.

4

Fail-closed enforcement

EmailOracleAuth on every OTP request, plus DLP/egress controls.

5

Public verifier UI

Compose hash, quote, and tx trail with a transparency log for every run.

This cohort can help make the project legible, verified, and real.

What we need

- Cryptographic critique of the leakage model and Theorem 1 assumptions.
- TEE / Flashbots / DevProof review of the deployment and provenance story.
- Tinker project entitlement, compute, and funding-rail support.

What it unlocks

- A verifiable end-to-end demo of private reward over sealed data.
- An ETH-native template for private benchmarks and data markets.
- Bio diligence without raw disclosure, with leakage counted up front.

wikigen.me

Help us make private rewards verifiable public infrastructure.

The goal is not to sell private data. It is to let private data prove, reward, validate, and settle under a boundary the ETH community can inspect.

Prime RL meets TEEs: environments become royalty-bearing assets, and inference pays their creators.

Shape Rotator project

Flashbots-aligned trust model

ETH-verifiable settlement